Lab Experiment 5

Aim :

To study and implement different addressing modes in C and analyse them using the GDB compiler/debugger.

Theory

Addressing modes define how operands are accessed during program execution. They are essential in understanding how memory and registers work internally.

1, Immediate Addressing:

The value is given directly in the instruction.
int a = 20;
The value **20** is directly assigned, no memory lookup needed. Fastest because no extra memory access.

2 . Direct Addressing:

The address of the variable is **directly known**.
int global_var = 50; The program directly accesses the memory location of `global_var`
Simple but depends on fixed memory location

3. Register Addressing:

Data is stored in **CPU registers**
int b = a;
Variable `a` is likely loaded into a register, then assigned to `b`.Very fast (registers are fastest memory).

4. Indirect Addressing:

Uses a pointer to access data.
int *p = &a;
Pointer `p` stores the address of `a`. Adds flexibility (used in dynamic memory, structures

5. Register Indirect Addressing:

Accessing data through pointer **dereferencing**
int c = *p;
CPU uses the address stored in `p` to get the value of `a`.Common in pointer-based operations

6. Indexed Addressing

Uses an index to access array elements.
int d = arr[2];
Address = base address + index offset

Useful for arrays and loops

7. Base Register Addressing

Uses a base address + offset.
int e = *(base + 3);`base`
points to start of array, `+3` moves forward.
Very important in memory calculations.

8. Relative Addressing

Used in **loops and branching**, relative to current instruction.
for(int i = 0; i < 3; I++)
CPU jumps relative to current position (used in loops).
Used internally for control flow.

9. Auto-Increment Addressing

Value is accessed, then index **increases automatically**.
f = arr[f++];
First `arr[0]` is accessed, then `f` becomes `1`.
Saves extra instruction.

9. Auto-Decrement Addressing

Value is accessed, then index de**creases automatically**.
g = arr[—g];
First `g` becomes `1`, then `arr[1]` is accessed.
Saves extra instruction.

Procedure :

Code

```c
#include <stdio.h>
// Direct Addressing (global variable)
int global_var = 50;

void function_example() {
    // Stack Addressing (local variables)
    int x = 5;
    int y = 10;

    // Immediate Addressing
    int a = 20;

    // Register Direct (normal variable use)
    int b = a;

    // Indirect Addressing
    int *p = &a;

    // Register Indirect
    int c = *p;

    // Indexed Addressing
    int arr[5] = {1,2,3,4,5};
    int d = arr[2];

    // Base Register Addressing (array + pointer)
    int *base = arr;
    int e = *(base + 3);

    // Relative Addressing (loop / control flow)
    for(int i = 0; i < 3; i++) {
        x += i;
    }

    // Auto-increment
    int f = 0;
    f = arr[f++];

    // Auto-decrement
    int g = 2;
    g = arr[--g];

    // Print to avoid optimization
    printf("%d %d %d %d %d %d %d\n", b, c, d, e, f, g, global_var);
}

int main() {
    function_example();
    return 0;
}
```

C program to implement various kinds of addressing modes

Output :

```
nitdelhi@pc:~/Desktop/241210035/COA$ nano addressing.c
nitdelhi@pc:~/Desktop/241210035/COA$ gcc -g addressing.c -o addressing
nitdelhi@pc:~/Desktop/241210035/COA$ gdb ./addressing
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./addressing...
(gdb) disassemble /s function_example
Dump of assembler code for function function_example:
addressing.c:
6       void function_example() {
   0x0000000000001169 <+0>:     endbr64
   0x000000000000116d <+4>:     push   %rbp
   0x000000000000116e <+5>:     mov    %rsp,%rbp
   0x0000000000001171 <+8>:     sub    $0x60,%rsp
   0x0000000000001175 <+12>:    mov    %fs:0x28,%rax
   0x000000000000117e <+21>:    mov    %rax,-0x8(%rbp)
   0x0000000000001182 <+25>:    xor    %eax,%eax

7           // Stack Addressing (local variables)
8           int x = 5;
   0x0000000000001184 <+27>:    movl   $0x5,-0x54(%rbp)

9           int y = 10;
   0x000000000000118b <+34>:    movl   $0xa,-0x4c(%rbp)

10
11          // Immediate Addressing
12          int a = 20;
   0x0000000000001192 <+41>:    movl   $0x14,-0x58(%rbp)

13
14          // Register Direct (normal variable use)
15          int b = a;
   0x0000000000001199 <+48>:    mov    -0x58(%rbp),%eax
   0x000000000000119c <+51>:    mov    %eax,-0x48(%rbp)

16
17          // Indirect Addressing
18          int *p = &a;
   0x000000000000119f <+54>:    lea    -0x58(%rbp),%rax
   0x00000000000011a3 <+58>:    mov    %rax,-0x30(%rbp)
```

Output of c program using gdb

```
22
23          // Indexed Addressing
24          int arr[5] = {1,2,3,4,5};
   0x00000000000011b0 <+71>:    movl   $0x1,-0x20(%rbp)
   0x00000000000011b7 <+78>:    movl   $0x2,-0x1c(%rbp)
   0x00000000000011be <+85>:    movl   $0x3,-0x18(%rbp)
   0x00000000000011c5 <+92>:    movl   $0x4,-0x14(%rbp)
   0x00000000000011cc <+99>:    movl   $0x5,-0x10(%rbp)

25          int d = arr[2];
   0x00000000000011d3 <+106>:   mov    -0x18(%rbp),%eax
   0x00000000000011d6 <+109>:   mov    %eax,-0x40(%rbp)
--Type <RET> for more, q to quit, c to continue without paging--c

26
27          // Base Register Addressing (array + pointer)
28          int *base = arr;
   0x00000000000011d9 <+112>:   lea    -0x20(%rbp),%rax
   0x00000000000011dd <+116>:   mov    %rax,-0x28(%rbp)

29          int e = *(base + 3);
   0x00000000000011e1 <+120>:   mov    -0x28(%rbp),%rax
   0x00000000000011e5 <+124>:   mov    0xc(%rax),%eax
   0x00000000000011e8 <+127>:   mov    %eax,-0x3c(%rbp)

30
31          // Relative Addressing (loop / control flow)
32          for(int i = 0; i < 3; i++) {
   0x00000000000011eb <+130>:   movl   $0x0,-0x50(%rbp)
   0x00000000000011f2 <+137>:   jmp    0x11fe <function_example+149>

33              x += i;
   0x00000000000011f4 <+139>:   mov    -0x50(%rbp),%eax
   0x00000000000011f7 <+142>:   add    %eax,-0x54(%rbp)

32          for(int i = 0; i < 3; i++) {
   0x00000000000011fa <+145>:   addl   $0x1,-0x50(%rbp)
   0x00000000000011fe <+149>:   cmpl   $0x2,-0x50(%rbp)
   0x0000000000001202 <+153>:   jle    0x11f4 <function_example+139>

34          }
35
36          // Auto-increment
```

Output of c program using gdb

```
34          }
35
36          // Auto-increment
37          int f = 0;
   0x0000000000001204 <+155>:   movl   $0x0,-0x38(%rbp)

38          f = arr[f++];
   0x000000000000120b <+162>:   mov    -0x38(%rbp),%eax
   0x000000000000120e <+165>:   lea    0x1(%rax),%edx
   0x0000000000001211 <+168>:   mov    %edx,-0x38(%rbp)
   0x0000000000001214 <+171>:   cltq
   0x0000000000001216 <+173>:   mov    -0x20(%rbp,%rax,4),%eax
   0x000000000000121a <+177>:   mov    %eax,-0x38(%rbp)

39
40          // Auto-decrement
41          int g = 2;
   0x000000000000121d <+180>:   movl   $0x2,-0x34(%rbp)

42          g = arr[--g];
   0x0000000000001224 <+187>:   subl   $0x1,-0x34(%rbp)
   0x0000000000001228 <+191>:   mov    -0x34(%rbp),%eax
   0x000000000000122b <+194>:   cltq
   0x000000000000122d <+196>:   mov    -0x20(%rbp,%rax,4),%eax
   0x0000000000001231 <+200>:   mov    %eax,-0x34(%rbp)

43
44          // Print to avoid optimization
45          printf("%d %d %d %d %d %d %d\n", b, c, d, e, f, g, global_var);
   0x0000000000001234 <+203>:   mov    0x2dd6(%rip),%esi        # 0x4010 <global_var>
   0x000000000000123a <+209>:   mov    -0x38(%rbp),%r8d
   0x000000000000123e <+213>:   mov    -0x3c(%rbp),%edi
   0x0000000000001241 <+216>:   mov    -0x40(%rbp),%ecx
   0x0000000000001244 <+219>:   mov    -0x44(%rbp),%edx
   0x0000000000001247 <+222>:   mov    -0x48(%rbp),%eax
   0x000000000000124a <+225>:   push   %rsi
   0x000000000000124b <+226>:   mov    -0x34(%rbp),%esi
   0x000000000000124e <+229>:   push   %rsi
   0x000000000000124f <+230>:   mov    %r8d,%r9d
   0x0000000000001252 <+233>:   mov    %edi,%r8d
   0x0000000000001255 <+236>:   mov    %eax,%esi
   0x0000000000001257 <+238>:   lea    0xda6(%rip),%rdi        # 0x2004
   0x000000000000125e <+245>:   mov    $0x0,%eax
   0x0000000000001263 <+250>:   callq  0x1070 <printf@plt>
   0x0000000000001268 <+255>:   add    $0x10,%rsp

46      }
   0x000000000000126c <+259>:   nop
   0x000000000000126d <+260>:   mov    -0x8(%rbp),%rax
   0x0000000000001271 <+264>:   xor    %fs:0x28,%rax
   0x000000000000127a <+273>:   je     0x1281 <function_example+280>
   0x000000000000127c <+275>:   callq  0x1060 <__stack_chk_fail@plt>
   0x0000000000001281 <+280>:   leaveq
   0x0000000000001282 <+281>:   retq
End of assembler dump.
(gdb) quit
```

Output of c program using gdb

Result:

Successfully implemented different addressing modes in C.Verified their working using GDB.Observed how variables are stored and accessed in memory, stack, and registers.Gained understanding of low-level program execution and memory handling.